

helixcode_vnv_execution_plan

HelixCode V&V Execution Plan — full test / Challenge / HelixQA cycle

Generated: 2026-06-24 · **Scope:** ordered, executable V&V plan for a freshly-booted infra run. **Authority:** §11.4.6 (data, no bluff), §11.4.3 (honest SKIP), §11.4.119 (single-resource-owner partitioning), §11.4.89 (background long tests). **Repo root:** /Volumes/T7/Projects/helix_code · **Inner app:** helix_code/ (run all make test* from here).

All commands below cite the file that defines them. Durations are classes, not measured.

0. Test-type catalogue (exact invocations)

Two Makefiles. Root Makefile = governance gates only. Inner helix_code/Makefile = real build/test.

Inner application — helix_code/Makefile

Step	Exact command (run from helix_code/)	Infra dep	Server?	Duration	Source
compile gate	make verify-compile (go build -tags=nogui ./...)	none	no	short	inner Makefile L319
unit (hermetic)	make test (go test -v ./..., cache-on)	none	no	medium	inner Makefile L107
unit (full, no-skip)	make test-unit-full (env .env.full-test, -count=1, 15m, ./internal/...)	test-infra-up	no	medium	inner Makefile L228
integration	make test-integration-full (-tags=integration -count=1 20m ./...)	test-infra-up	PG/Redis/Ollama	long	inner Makefile L234
				long	

Step	Exact command (run from helix_code/)	Infra dep	Server?	Duration	Source
e2e challenges	make test-e2e-full → cd tests/e2e/ challenges && go run cmd/runner/ main.go -all	test- infra- up	live server :8080		inner Makefile L240
security	make test- security-full (-count=120m ./ tests/security/...)	test- infra- up	some	long	inner Makefile L246
load	make test-load- full (./test/ load/... 30m)	test- infra- up	live server	long	inner Makefile L252
stress+chaos (unit)	make stress-chaos (-race -run 'Stress\ Chaos' across ~35 internal pkgs)	none	no	long	inner Makefile L114
stress+chaos meta	make stress-chaos- meta (-run TestMeta)	none	no	medium	inner Makefile L121
stress+chaos infra	make stress-chaos- infra (-tags=integration -race, real PG+Redis)	test- infra- up	PG/ Redis	long	inner Makefile L132
coverage	make test-coverage (-race -coverprofile)	none	no	long	inner Makefile L146
benchmark	make test- benchmark	none	no	long	inner Makefile L157
everything, zero-skip	make test-full (unit + integration, env- loaded, -count=1 30m)	test- infra- up	yes	very long	inner Makefile L220
all types in sequence	make test-complete (unit-full → integration-full → e2e- full → security-full → load-full → benchmark → verifier-challenges)	test- infra- up	yes	very long	inner Makefile L258
verifier unit	make test- verifier-unit (-count=1 ./ internal/ verifier/...)	none	no	medium	inner Makefile L274
verifier integration	make test- verifier- integration	verifier svc :8081	verifier	medium	inner Makefile L280

Step	Exact command (run from <code>helix_code/</code>)	Infra dep	Server?	Duration	Source
	(-tags=integration) — needs verifier @ :8081				
verifier challenges	make test-verifier-challenges (hardcode + capability grep gates)	none (cap needs verifier)	no	short	inner Makefile L286

-count=1 is load-bearing on every *-full / verifier-* target: real-infra runs must re-execute (cached PASS = CONST-035 false-success). Plain `make test` keeps cache (hermetic).

Root governance gates — Makefile

`make verify-compile` is inner. Root: `make no-silent-skips, make verify-governance-cascade, make verify-foundation (composite), make bluff-detector, make scan-secrets-root`. These are pre-flight gates, not the runtime V&V cycle — run them first but they need no infra.

1. Infra boot — two distinct mechanisms

Mechanism	What it boots	Use for
make test-infra-up (from <code>helix_code/</code>)	<code>docker-compose.full-test.yml</code> via auto-detected <code>docker compose/podman compose</code> (inner Makefile L194). Boots: postgres:16 (:5432, db <code>helixcode_test</code> , user <code>helixcode</code> , pwd <code>helixcode_test_password</code>), redis:7 (:6379, no pwd), <code>memcached</code> (:11211), ollama (:11434, pulls <code>llama2:7b</code>), mock-llm-server (:8090 — OpenAI/Anthropic/etc. all point here), <code>selenium-chrome</code> (:4444), <code>chromedp</code> (:9222), <code>ssh-server+3 ssh-workers</code> (:2222-2225), <code>pulseaudio</code> , <code>cognee</code> (:8000), <code>weaviate</code> , <code>chromadb</code> (:8082), <code>qdrant</code> (:6333), <code>mock-slack</code> (:8091), <code>prometheus</code> (:9090), <code>grafana</code> (:3000), helixcode server (:8080) . Sleeps 30s for health.	the test/Challenge/HelixQA cycle
./helix start (repo root facade)	<code>docker-compose.helix.yml</code> (the standalone <i>product</i> stack —	running the product standalone

Mechanism	What it boots	Use for
	single helixcode container exposing API :8080, SSH :2222, Web :3000). NOT the test stack.	

“Distribute by starting the helixcode main binary”: the test stack’s helixcode service (compose L461, port 8080:8080) IS bin/helixcode = cmd/server. It depends on PG+Redis+Ollama being healthy first — test-infra-up brings them up together. The e2e runner, load tests, and all api/web HelixQA banks hit http://localhost:8080, so the server MUST be up + healthy before those steps. Env for server-side test runs comes from .env.full-test (set -a && . ./env.full-test), which is what every *-full target sources.

Credentials note: .env.full-test uses **mock** keys for ALL cloud LLM providers (OPENAI_API_KEY=mock-..., base → http://localhost:8090). Real cloud-provider generation is NOT exercised by this stack — only Ollama (real, local) + mock-llm-server. Tests asserting real cloud-provider wire behaviour are honest-SKIP candidates (no real key present).

Teardown: make test-infra-down (down -v, drops volumes).

2. Challenges

Two distinct Challenge surfaces:

- 1. Inner e2e challenges** — helix_code/tests/e2e/challenges/cmd/runner/main.go.
 - Run: cd helix_code/tests/e2e/challenges && go run cmd/runner/main.go -all
 - Or via make test-e2e-full (sources env + same command).
 - Flags (main.go L20-36): -challenge <id>, -interfaces cli,tui,rest,websocket (default cli), -distributions single,worker_2,worker_5,worker_10, -providers ollama, -models llama2, -max-concurrent 3 (default), -timeout 45m, -list, -export-report <file>, -results-dir, -logs-dir.
 - Definitions (definitions/, 6 challenges): ascii-art-generator, cli-task-manager, json-validator-cli, notes-project, tic-tac-toe-tui, url-shortener.
 - **Infra:** needs live server :8080 + Ollama (default provider ollama/llama2). These drive real LLM generation through the server.
- 2. submodules/challenges** (vasic-digital/Challenges) — Go module with its own Makefile.
 - cd submodules/challenges && make qa-all (= build + vet + test), make challenge, make new-capabilities (= challenge-agentic-subagents + challenge-persistent-memory), make anti-bluff (scan + anchors + mutation).
 - Banks under banks/ (examples, yole); fixtures/baselines under challenges/.
 - **Infra:** make test/test-race hermetic; make test-integration + the challenge-* capability runs may need the live HelixCode stack.

3. HelixQA — submodules/helix_qa (HelixDevelopment/HelixQA)

The autonomous QA framework. Binary cmd/helixqa with subcommands (main.go L56-72): **run** (multi-platform pipeline), **list**, **report**, **autonomous**, **http** (run a bank's http: cases against a live server, NO browser/LLM — main.go L97), **replay**, **signoff**, **version**.

Invocation (canonical, from inner Makefile L722 helixqa-challenge)

```
cd submodules/helix_qa && go run ./cmd/helixqa run \
  --banks ./banks/full-qa-api,./banks/full-qa-web \
  --platform api,web \
  --browser-url http://localhost:8080 \
  --output ./qa-results/helixcode-<ts> \
  --report markdown,html,json --validate --tickets
```

run flags (main.go L116-150): **--banks** <comma-paths> (required), **--platform** android|web|desktop|all (default all), **--browser-url**, **--output**, **--report**, **--validate**, **--tickets**. > NOTE the inner Makefile's HELIXQA_DIR := ../HelixQA path is the legacy flat layout; the actual submodule is at submodules/helix_qa/. Run the go run ./cmd/helixqa ... command from submodules/helix_qa/ directly (do NOT rely on make helixqa-challenge's ../HelixQA path unless that sibling exists).

Test BANKS (submodules/helix_qa/banks/, ~120 .yaml/.json)

HelixCode-targeted banks (the relevant ones for this run), grouped by platform need:

- **api (server-only, parallel-safe via http subcommand — no browser/LLM):** helixcode-auth.yaml, helixcode-system.yaml, helixcode-health-readiness.yaml, helixcode-cors-middleware.yaml, helixcode-static-routes.yaml, helixcode-protected-subresources.yaml, helixcode-task-workflow.yaml, helixcode-worker-management.yaml, helixcode-mcp.yaml, helixcode-mcp-fs.yaml, helixcode-lsp.yaml, helixcode-memory-systems.yaml, helixcode-persistent-memory.yaml, helixcode-providers-verifier.yaml, helixcode-skills.yaml, helixcode-plugins.yaml, helixcode-ensemble-members.yaml, helixcode-streaming.yaml (api + tui), full-qa-api.yaml.
- **web (needs selenium :4444 / chromedp :9222 + server :8080):** full-qa-web.yaml, helixcode-qa-screenshot.yaml.
- **tui:** helixcode-streaming.yaml (tui cases), tui-recording-validation.yaml.
- **mobile / other-platform (NOT HelixCode core — SKIP candidates here):** full-qa-android*, full-qa-androidtv*, nexus-mobile-ios.yaml, nexus-desktop-*, capture-android.yaml, plus atmosphere*, boba*, openclawing2-*, yole-*, nexus-* (sibling-project banks).

Server-only fast path (no browser, no LLM): go run ./cmd/helixqa http --banks <api-banks> --browser-url http://localhost:8080 — runs the http: cases against the live server.

Build/test the framework itself: `cd submodules/helix_qa && make build / make test / make qa-all (= challenge + anti-bluff).`

HelixCode-side xiaomi bank

`helix_code/qa-integration/xiaomi_test_bank.yaml` + `helix_code/tests/e2e/challenges/xiaomi_provider_challenge.go` — a provider-specific challenge (real Xiaomi LLM provider). **Needs a real Xiaomi API credential** → honest §11.4.3 SKIP if absent.

4. ORDERED execution plan (boot → unit → integration → e2e → Challenges → HelixQA → security/perf)

Legend: **[serial-server]** = hammers the single `:8080` server, **MUST** be the exclusive driver (§11.4.119); **[parallel-safe]** = no shared exclusive resource; **[bg]** = run backgrounded per §11.4.89.

#	Step	Command	Infra	§11.4.119
0a	Governance pre-flight	(root) <code>make verify-foundation && make verify-governance-cascade</code>	none	parallel-safe
0b	Compile gate	(helix_code/) <code>make verify-compile</code>	none	parallel-safe
1	Unit (hermetic)	(helix_code/) <code>make test</code>	none	parallel-safe [bg]
1b	Verifier unit	(helix_code/) <code>make test-verifier-unit</code>	none	parallel-safe [bg]
2	Boot infra	(helix_code/) <code>make test-infra-up</code> (waits 30s; pulls llama2:7b); then <code>make test-infra-status</code> to confirm PG/Redis/Ollama/ helixcode :8080 healthy	docker/ podman	—
3	Integration	(helix_code/) <code>make test-integration-full</code>	PG/Redis/ Ollama	mostly parallel-safe (own DB); avoid concurrent with server-driving steps
4		<code>make stress-chaos</code> (none); <code>make stress-</code>		unit parallel-

#	Step	Command	Infra	§11.4.119
	Stress+chaos (unit then infra)	chaos-infra (PG+Redis)	none / PG+Redis	safe; infra serialize w/ #3 on DB
5	E2E challenges	(helix_code/) make test-e2e-full (server :8080 + Ollama)	full stack	[serial-server] — drives :8080 + Ollama generation qa-all parallel-safe; capability runs [serial-server] if they hit :8080
6	Challenges submodule	cd submodules/ challenges && make qa-all ; capability: make new-capabilities	hermetic / live stack	
7	HelixQA api banks (fast, server-only)	cd submodules/ helix_qa && go run ./cmd/helixqa http --banks ./banks/helixcode-auth.yaml,...,./banks/full-qa-api.yaml --browser-url http://localhost:8080	server :8080	[serial-server] (all share :8080 — run as ONE invocation, not concurrent)
8	HelixQA full run (api+web+validate+tickets)	cd submodules/ helix_qa && go run ./cmd/helixqa run --banks ./banks/full-qa-api.yaml,./banks/full-qa-web.yaml --platform api,web --browser-url http://localhost:8080 --output ./qa-results/helixcode-<ts> --report markdown,html,json --validate --tickets	server :8080 + selenium :4444 / chromedp :9222	[serial-server] — exclusive over :8080 AND the single browser session
9	HelixQA framework self-test	cd submodules/ helix_qa && make qa-all	hermetic	parallel-safe [bg]
10	Security	(helix_code/) make test-security-full ; root scanners make scan-gosec	full stack / none	server-touching parts [serial-

#	Step	Command	Infra	§11.4.119
		scan-trivy (need no infra)		server]; static scanners parallel- safe [bg]
11	Load / perf	(helix_code/) make test-load-full; make test-benchmark (none)	server :8080 / none	load [serial- server]; benchmark parallel- safe [bg]
12	Verifier integration	(helix_code/) make test-verifier- integration	verifier @ :8081	parallel- safe (own service)
13	Teardown	(helix_code/) make test-infra-down	—	—

§11.4.119 hard rule: Steps 5, 7, 8, 10(server), 11(load) all exclusively drive the single :8080 server (and steps 8 the single browser). They MUST be **serialized** — exactly one server-driving step at a time. Parallel-safe [bg] steps (1, 1b, 9, 10-scanners, 11-benchmark, 12) MAY run concurrently with a server-driving step since they touch no shared exclusive resource. Per §11.4.89, long steps (3, 5, 8, 10, 11) run via nohup ... > qa-results/<id>.log 2>&1 & disown.

Alternative one-shot: make test-complete runs unit-full → integration-full → e2e-full → security-full → load-full → benchmark → verifier-challenges in sequence (already serial; inner Makefile L258) — but it does NOT include the HelixQA banks (steps 7-9) or the Challenges submodule (step 6), which must be run separately.

5. Honest §11.4.3 SKIP candidates (credential/SDK the System likely lacks)

Item	Why it SKIPs	Needed to un-SKIP
Real cloud-LLM provider generation (OpenAI/Anthropic/Gemini/Groq/Mistral/Cohere/Azure/AWS Bedrock)	.env.full-test ships mock keys → :8090 mock-llm-server; no real wire calls	real API keys in a non-mock .env
xiaomi_provider_challenge.go + qa-integration/xiaomi_test_bank.yaml	needs real Xiaomi provider credential	real Xiaomi key
Verifier integration (make test-verifier-integration)	needs LLMsVerifier service @ http://localhost:8081 (NOT in full-test)	run the verifier service

Item	Why it SKIPs	Needed to un-SKIP
	compose — cognee uses :8081→8080 host map; confirm verifier separately)	
HelixQA mobile/TV banks (full-qa-android*, nexus-mobile-ios, capture-android)	need real Android/iOS device or emulator/simulator + ADB/Xcode SDK	device + platform SDK
HelixQA desktop banks (nexus-desktop-macos/windows/linux)	need the desktop GUI build on that OS	per-OS desktop binary
Security scanners make scan-snyk / make scan-sonarqube	need SNYK_TOKEN / SONAR_TOKEN in helix_code/.env	tokens (open-source scanners gosec/trivy/grype run without)
Sibling-project HelixQA banks (atmosphere*, boba-*, openclawing2-*, yole-*)	not HelixCode — different system under test	out of scope for HelixCode run

6. Pre-flight verification before claiming any PASS (§11.4.6)

1. make test-infra-status — confirm PG, Redis, Ollama, and **helixcode** :8080 all healthy (the server is the §11.4.119 shared resource).
2. curl -fsS http://localhost:8080/health (or /readyz) returns 200 before any server-driving step (5/7/8/10/11).
3. curl -fsS http://localhost:11434/api/tags lists llama2:7b before the e2e challenges (provider ollama).
4. Every PASS carries a captured-evidence artefact path (qa-results/ log, report file) — no metadata-only PASS.